Proving Data Structures Correct: Abstract Data Types

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Implementing ADTs

1. Implement interface
2. Prove specification

Example

Binary search trees from the Week 4 sheet (Exercises 4.1–4.3):
functions `isin` and `ins` on type `'a tree`, abstraction function `tree_set`, invariant `bst`.

Now implement locale `Set`:

©Mohammad Abdulaziz

In Isabelle: **interpretation**

interpretation `BST: Set`
where `empty = Leaf` **and** `isin = isin`
  and `insert = ins` **and** `set = tree_set`
  and `invar = bst`
proof
  show `tree_set Leaf = {}` ⟨proof⟩
next
  fix `s` **assume** `bst s`
  show `tree_set (ins x s) = tree_set s ∪ {x}`
  ⟨proof⟩
next
⋮
qed

The cases are the Week 4 lemmas `tree_set_isin`, `tree_set_ins` and `bst_ins`.

Implementing `Set` with 2-3 Trees

interpretation `T23: Set`
where `empty = Leaf` **and** `isin = isin`
and `insert = insert` **and** `set = set_tree23`
and `invar = (λt. bst23 t ∧ complete t)`

Proof obligations:

- `isin`, `insert`, `bst23`: like the Week 4 BST lemmas, with more cases
- new: `complete t ⟹ complete (insert x t)` (Week 7)

Same interface, two implementations

©Mohammad Abdulaziz

Each interpretation of `Set` also yields

- functions

```
BST.set_of_list :: 'a list ⇒ 'a tree
T23.set_of_list :: 'a list ⇒ 'a tree23
```

- theorems

```
bst (BST.set_of_list xs)
bst23 (T23.set_of_list xs) ∧
  complete (T23.set_of_list xs)
```

©Mohammad Abdulaziz